Міністерство освіти та науки України

Національний технічний університет України “Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи №2

з дисципліни “Розробка мобільних застосувань під Android”

Виконав:

Студент 3 курсу гр.

ІП-33

Бресків Д. А.

Перевірив:

Орленко С. П.

“26” лютого 2026

2026

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набути практичні навички з використання фрагментів для інтерфейсу користувача.

ЗАВДАННЯ

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («OK»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

Посилання на відео-демонстрацію готового застосунку:
https://drive.google.com/file/d/1esYV7BQFBG24FXlfk4BQ9bPMlMQy0Fyk/view?usp=sharing

КОНТРОЛЬНІ ПИТАННЯ

1. Призначення та можливості компоненту Фрагмент

Фрагмент — це частина інтерфейсу та логіки в межах Activity, яку можна багаторазово використовувати. Він дозволяє будувати екран з кількох незалежних частин, гнучко підлаштовувати UI під різні розміри екрана, а також динамічно додавати, прибирати або замінювати частини інтерфейсу під час роботи застосунку. Фрагменти мають власний життєвий цикл, можуть

мати власні ресурси та обробники подій, і можуть зберігати стан при зміні конфігурації разом з Activity.

2. Життєвий цикл компонента Фрагмент

Основні етапи життєвого циклу фрагмента: приєднання до Activity, створення, створення інтерфейсу, перехід у активний стан, пауза, зупинка, знищення інтерфейсу, знищення фрагмента, від'єднання від Activity. Типові методи: onAttach, onCreate, onCreateView, onViewCreated, onStart, onResume, onPause, onStop, onDestroyView, onDestroy, onDetach. Важливо, що інтерфейс фрагмента може бути знищений окремо від самого об'єкта фрагмента, тому onDestroyView використовується для звільнення ресурсів, пов'язаних з UI.

3. Способи створення компонента Фрагмент

Фрагмент можна створити як клас, що наслідує Fragment, і реалізувати створення інтерфейсу через onCreateView або використати конструктор з ресурсом розмітки. Також поширений підхід — створення фрагмента через фабричний метод newInstance, який приймає параметри та записує їх у arguments, щоб правильно відновлювати стан після пересоздання. Фрагмент може мати XML-розмітку або створювати UI програмно.

4. Способи управління компонентом Фрагмент

Керування фрагментами здійснюється через FragmentManager та операції FragmentTransaction. Основні дії: додати фрагмент, замінити фрагмент, прибрати фрагмент, показати або сховати фрагмент. Зміни застосовуються через commit або commitNow, а для підтримки навігації назад можна додавати транзакції у back stack. Також можна знаходити фрагменти за ідентифікатором контейнера або тегом і керувати їхнім станом.

5. Способи взаємодії між Фрагментами

Рекомендований спосіб взаємодії — через спільну Activity або спільний ViewModel, де один фрагмент змінює дані, а інший спостерігає за ними. Також

використовують інтерфейси-колбеки, які реалізує Activity, щоб фрагмент повідомляв про подію, а Activity передавала її іншому фрагменту. Для передачі даних між фрагментами можуть застосовуватись arguments, але для подій і стану зручніші ViewModel або інші спостережувані механізми.

6. Поняття системи, малої системи та мобільної платформи

Система — це сукупність взаємопов'язаних елементів, які працюють разом для досягнення спільної мети. Мала система — це система з відносно невеликою кількістю елементів і простішими зв'язками між ними, яку легше описати, реалізувати та протестувати. Мобільна платформа — це апаратно-програмне середовище для мобільних пристроїв, яке включає операційну систему, набір сервісів, засоби розробки та правила виконання застосунків.

7. Типи мобільних застосунків

Розрізняють нативні, веб-застосунки та гібридні застосунки. Нативні створюються під конкретну платформу і мають найкращу продуктивність та доступ до можливостей пристрою. Веб-застосунки працюють у браузері і залежать від веб-технологій. Гібридні поєднують веб-інтерфейс з контейнером у вигляді застосунку і можуть використовувати частину можливостей пристрою через спеціальні бібліотеки. Також виділяють кросплатформні застосунки, які пишуться одним кодом і збираються під різні платформи.

8. Класифікація та загальна характеристика середовищ розробки мобільних застосунків

Середовища розробки можна поділити на нативні та кросплатформні. Нативні орієнтовані на одну платформу і забезпечують повну інтеграцію з її інструментами, емуляторами, профайлерами та системами збірки. Кросплатформні дозволяють розробляти застосунки для кількох платформ одночасно, використовуючи спільну кодову базу, але можуть мати обмеження у доступі до специфічних можливостей і продуктивності. Також існують

середовища для веб-застосунків, які базуються на браузерних технологіях і фреймворках.

9. Класифікація та загальна характеристика мобільних платформ

Мобільні платформи класифікують за типом операційної системи та екосистеми: відкриті та більш закриті. Платформи відрізняються моделлю безпеки, механізмами розповсюдження застосунків, набором системних сервісів і API, а також інструментами розробки. Також платформи можна класифікувати за типом пристроїв і застосування: смартфони, планшети, носимі пристрої, автомобільні системи, телевізійні платформи. Кожна платформа має власні правила інтерфейсу, життєвого циклу застосунків і вимоги до сумісності.